# *3d.* Variable Scope & Relevance of Functions

Martin Alfaro
PhD in Economics

## INTRODUCTION

**Variable scope** refers to the code block in which a variable is accessible. The concept allows us to distinguish between **global variables**, which are accessible in any part of the code, and **local variables**, which are confined to specific blocks like functions or loops. The existence of scopes determines that the same variable `x` could refer to different objects, depending on where it's called.

When it comes to functions, Julia adheres to specific rules for variable scope. Specifically, given a variable `x` defined outside a function:

- If a new variable `x` is defined inside a function or is passed to a function as an argument, this `x` is considered *local* to that function. Moreover, any reference to `x` within the function refers to this local variable, with no connection to the `x` defined outside the function.
- If a function neither defines a new `x` nor `x` is a function argument, references to `x` inside the function point to the variable defined outside the function (i.e., the global `x`).

In this section, we'll show how these rules work in practice.

## GLOBAL AND LOCAL VARIABLES

A variable that's local to a function exists solely within that function's scope. This means that, once the function finishes executing, these variables cease to exist. Consequently, any attempt to reference them outside the function will raise an error.

Variables local to a function encompass:

1. the function arguments,
2. the variables defined in the function body.

Any other variable in a function that's not *i)* or *ii)* necessarily refers to a global variable.

Recognizing whether a variable is local or global is crucial for predicting how a program behaves. Indeed, a local variable may share the same name as a global one, without them being related. The following examples help clarify the differences between global and local variables in practice.

**LOCAL VARIABLES**

```
x = "hello"

function foo(x)         # 'x' is local, unrelated to 'x = hello' above
    y = x + 2           # 'y' is local, 'x' refers to the function argument

    return x,y
end
```

```
julia> foo(1)
1                   # local x
3                   # local y
julia> x
"hello"
julia> y
ERROR: UndefVarError: y not defined
```

**GLOBAL VARIABLES**

```
z = 2

function foo(x)
    y = x + z           # 'x' refers to the function argument, 'z' refers to the global

    return x,y,z
end
```

```
julia> foo(1)
1                   # local x
3                   # local y
2                   # global z
julia> x
ERROR: UndefVarError: x not defined
julia> z
2
```

## THE ROLE OF FUNCTIONS

In good programming practice, **functions** are best conceived **as self-contained mini-programs**, each designed to perform a specific and well-defined **task**. When a function successfully captures the implementation of that task, it becomes reusable: we can apply the same operation to different inputs or objects, thus avoiding code duplication. This reusability is one of the key reasons functions are such powerful abstractions. By defining a task once, we can rely on it consistently throughout a program, which not only reduces redundancy but also makes the overall structure easier to maintain and reason about.

To achieve this, a function must clearly express both the logic required to perform the task and the data necessary to accomplish it. Within this perspective, local variables simply act as temporary placeholders that help articulate the mechanics of that task. [1] Since their meaning is tied exclusively to the function's internal process, it's natural that local variables can't be accessed from outside. Moreover, this ensures that the function only interacts with the rest of the program through its inputs and outputs, thereby preserving its integrity as a self-contained well-defined task.

To illustrate this view of functions, consider a variable `x`, along with another variable `y` computed by transforming `x` through a function `f`. In particular, assume a transformation that doubles `x`, so that `y = 2 * x`. The following are two approaches to calculating `y`.

**APPROACH 1**

```
x          = 3

double()   = 2 * x
y          = double()
```

**APPROACH 2**

```
x          = 3

double(x)  = 2 * x
y          = double(x)
```

**APPROACH 2 - EQUIVALENT**

```
x          = 3

double(🐒) = 2 * 🐒
y          = double(x)
```

The function in Approach 1 relies on the global variable `x`. This practice is highly discouraged for several reasons. Firstly, it prevents the reusability of the function, as it's specifically designed to double the global variable `x`, rather than acting as a mini-program that doubles *any* variable.

Second, the inclusion of the global variable `x` compromises the function's self-containment, as the function's output depends on the value of `x` at the moment of execution. If you work on a long project, this will turn the code prone to bugs.

Lastly, global variables have a detrimental impact on performance, a topic we'll study later in the book. In fact, global variables in Julia are a direct performance killer.

In contrast, Approach 2 refers to `x` as a local variable. This `x` is unrelated to the global variable `x`—it simply serves as a label to identify the variable to be doubled. Indeed, we could've replaced `x` with any other label, as demonstrated in Approach 3 through the monkey emoji, `🐒`.

By avoiding referencing any variable outside its scope, Approach 2 makes the function self-contained. This allows users to easily anticipate the consequence of executing `double` through a simple inspection of the function, eliminating the need to review the entire codebase. Thus, Approach 2 aligns with the interpretation of a function as a self-contained mini-program: the function embodies the task of doubling a variable, turning the function reusable and applicable to any variable. In this context, applying `double` to the global variable `x` becomes just one possible application.

# RECOMMENDATIONS FOR THE USE OF FUNCTIONS

Structuring code around functions offers numerous advantages. However, to fully realize these benefits, users must adhere to certain principles when writing code. This section outlines a few of them and should be considered as a mere introduction to the subject. The topic will be investigated further when we explore high performance.

## AVOID GLOBAL VARIABLES IN FUNCTIONS

Global variables are strongly discouraged. This is not only due to the reasons mentioned previously, but also because they have a devastating impact on performance in Julia. The easiest solution to this issue is to pass global variables as function arguments. This practice will actually become second nature once you start viewing functions as self-contained mini-programs. Specifically, by adopting this perspective, you'll conceive local variables as labels to describe a task, rather than references to global variables. This shift in mindset can help you write more efficient and maintainable code.

## AVOID REDEFINING VARIABLES WITHIN FUNCTIONS

The suggestion applies to both local variables and function arguments. Redefining these variables can have several disadvantages, including reduced code readability and potential performance degradation. Therefore, it's recommended that you define new variables instead of redefining existing ones. This approach is demonstrated in the following example.

**NOT RECOMMENDED**

```
function foo(x)
   x      = 2 + x          # redefines the argument

   y      = 2 * x
   y      = x + y          # redefines a local variable
end
```

**PREFERRED**

```
function foo(x)
   z      = 2 + x           # new variable

   y      = 2 * x
   output = z + y           # new variable
end
```

**(OPTIONAL)** - Another Issue of Redefining Variables

## MODULARITY

We've emphasized the importance of viewing functions as self-contained mini-programs, designed to perform specific tasks. This perspective leads us to highlight the importance of **modularity**: the practice of breaking down a program into multiple small functions, each with its own distinct purpose, inputs, and outputs.

The primary benefit of modularity is the ability to work with independent code blocks. By keeping these blocks separate, we can decompose complex problems into multiple manageable tasks, making it easier to test and debug code. Additionally, modularity makes it possible to eventually improve or substitute parts of the code, without breaking the entire program.

A helpful way to understand this principle is by considering the analogy of building a Lego minifigure. In the first step, multiple blocks are created independently, each representing a specific part of the figure, such as the legs, torso, arms, and head. Then, in the second stage, these individual blocks are brought together and assembled into an integrated minifigure.

This two-step approach offers several advantages. By focusing on each block individually, we can concentrate and refine each part without worrying about the entire structure. Additionally, it provides great flexibility: since each block is created independently, we can modify specific blocks without having to rebuild the entire figure. For instance, if we want to change the figure's head, we can simply swap out the corresponding block, without starting from scratch.

The principle of modularity is closely tied to the suggestion of writing short functions. Some proponents even argue that functions should be limited to fewer than five lines of code Indeed, entire books have been written based on this principle. Although this viewpoint may be considered rather extreme, it clearly emphasizes the advantages of avoiding lengthy functions.

**(OPTIONAL)** - Example of Modularity

---

FOOTNOTES

[1] Local variables play a similar role to integration variables in math. Formally, $\mathrm{d}t$ in $\int f\,(t)$ for some function $f$ is simply a symbol indicating over which variable we're integrating. The integral could be equivalently expressed using any other integration variable, such as $x$ in $\int f\,(x)\,\mathrm{d}x$.